Inheritance:

Inheritance is the capability of one class to acquire properties and characteristics from another class.

The class whose properties are inherited by other class is called the **Parent** or **Base** or **Super** class. And, the class which inherits properties of other class is called **Child** or **Derived** or **Sub** class.

Inheritance makes the code reusable. When we inherit an existing class, all its methods and fields become available in the new class, hence code is reused.

Members that cannot be inherited from parent class:

1. Private members of a Parent class.

2. Constructors and Destructor of a Parent class.

3. Friend functions and friend classes of a Parent class.

Advantages of Inheritance:

1. Code Reusability

2. Method overriding

Basic Syntax of Inheritance:

```
class Subclass_name : MOI Superclass_name
```

While defining a subclass like this, the super class must be already defined or atleast declared before the subclass declaration.

MOI(Mode of inheritance) or visibility is used to specify, the mode in which the properties of superclass will be inherited into subclass, public, private or protected.

Vector India

Example :

```
#includeiostream>

using namespace std;

// Base class
class Shape {
   public:
      void setWidth(int w) {
         width = w;
      }
      void setHeight(int h) {
         height = h;
      }

   protected:
      int width;
      int height;
};

// Derived class
class Rectangle: public Shape {
   public:
      int getArea() {
         return (width * height);
      }
};

int main(void) {
   Rectangle Rect;
   Rect.setWidth(5);
   Rect.setHeight(7);

   // Print the area of the object.
   cout << "Total area: " << Rect.getArea() << endl;

   return 0;
}
```

Vector India

```
Output:

Total area: 35.
```

Mode of Inheritance or Visibility Mode:

Depending on Visibility mode used while inheritance, the availability of class members of Super class in the sub class changes. It can either be private, protected or public.

1) Public Inheritance

This is the most used inheritance mode. In this the protected member of super class becomes protected members of sub class and public becomes public.

```
class Subclass : public Superclass
```

2) Private Inheritance

In private mode, the protected and public members of super class become private members of derived class.

```
class Subclass : Superclass   // By default its private inheritance
```

3) Protected Inheritance

In protected mode, the public and protected members of Super class becomes protected members of Sub class.

```
class subclass : protected Superclass
```

Vector India

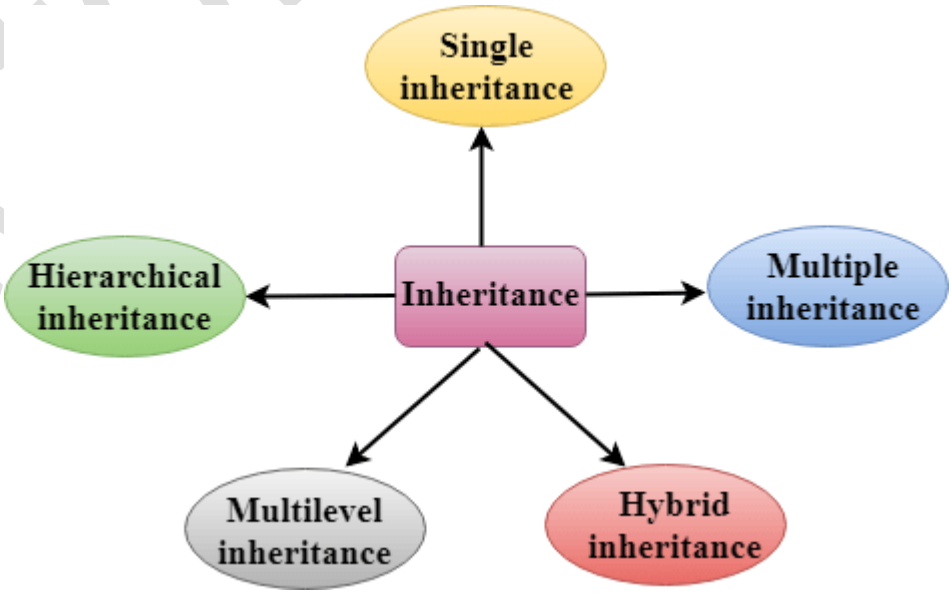
Table showing all the Visibility Modes

Base class	Derived Class Public Mode	Derived Class Private Mode	Derived Class Protected Mode
Private	Not Inherited	Not Inherited	Not Inherited
Protected	Protected	Private	Protected
Public	Public	Private	Protected

Types of Inheritance

C++ supports five types of inheritance:

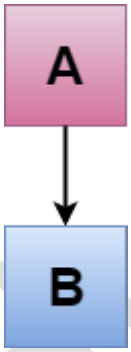
- Single inheritance
- Multiple inheritance
- Hierarchical inheritance
- Multilevel inheritance
- Hybrid inheritance



Vector India

C++ Single Inheritance

Single inheritance is defined as the inheritance in which a derived class is inherited from the only one base class.



Where 'A' is the base class, and 'B' is the derived class.

C++ Single Inheritance Example:

When one class inherits another class, it is known as single inheritance.

```
#include<iostream>
using namespace std;
class Person
{
      protected : string name;
                  int age;
      public : void setName()
               {
                     cout<<”enter the name”<<endl;
                     cin>>name;
               }
               void setAge()
               {
```

Vector India

```
                cout<<"enter the age:"<<endl;
                cin>>age;
        }
};
class Faculty : public Person
{       string subject;
        public: void setSubject()
                {
                cout<<"enter the subject:"<<endl;
                cin>>subject;
                }
                void printDetails()
                {
                cout<<"name:"<<name<<endl;
                cout<<"age:"<<age<<endl;
                cout<<"subject:"<<subject<<endl;
                }
};
main()
{
        Faculty obj;
        obj.setName();
        obj.setAge();
        obj.setSubject();
        obj.printDetails();
}
```

Vector India

Output:

```
enter the name:
        Rahul
enter the age:
        23
enter the subject:
        C++
name: Rahul
age:   23
subject : C++
```

Multilevel Inheritance

Multilevel inheritance is a process of deriving a class from another derived class



Vector India

Multi Level Inheritance Example

When one class inherits another class which is further inherited by another class, it is known as multi level inheritance in C++. Inheritance is transitive so the last derived class acquires all the members of all its base classes.

Let's see the example of multi level inheritance in C++.

```
#include<iostream>

using namespace std;
class Animal{
        public:
        void eat()
        {
                cout<<"Eating..."<<endl;
        }
};
class Dog : public Animal
{
        public:
                void bark(){
                        cout<<"Barking..."<<endl;
                }
};
class BabyDog: public Dog
{
        public:
                void weep(){
                cout<<"Wee       ping...";
        }
};
int main( ){
                BabyDog d1;
                d1.eat();
                d1.bark();
                d1.weep();
                return 0;
}
```

Vector India

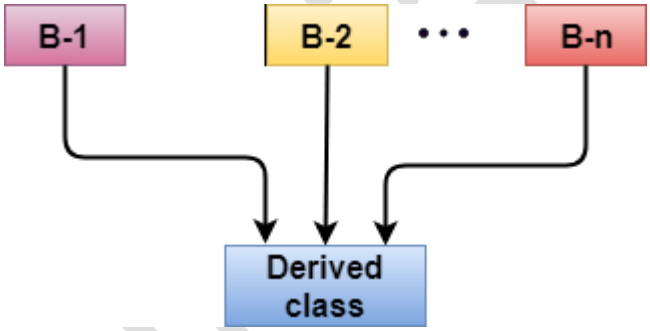
Output:

```
Eating...

Barking...
Weeping...
```

C++ Multiple Inheritance

Multiple inheritance is the process of deriving a new class that inherits the attributes from two or more classes.



Syntax of the Derived class:

class D : visibility B-1, visibility B-2

```
{
// Body of the class;
}
```

Example :

```
#include<iostream>
using namespace std;
        class A
        {   protected:
                int a;
            public:
```

```
        void get_a(int n)
        {
                a=n;
        }
};
class B
{
        protected:
                int b;
        public:
                void get_b(int n)
                {
                        b=n;
                }
        };
class C : public A,public B
{
public:
        void display()
        {
                cout<<"The value of a is : "<<a<<endl;
                cout<<"The value of b is :"<<b<<endl;
                cout<<"Addition of a and b is :"<<a+b<<endl;
}
};
int main()
{
        C c;
        c.get_a(10);
        c.get_b(20);
        c.display();
        return 0;
}
```

Output:

```
The value of a is : 10
The value of b is : 20
```

Vector India

```
Addition of a and b is : 30
```

Ambiquity Resolution in Inheritance

Ambiguity can be occurred in using the multiple inheritance when a function with the same name occurs in more than one base class.

Let's understand this through an example:

```
#include<iostream>
using namespace std;
class A
{
	public:
		void display()
	{
		cout<<"Class A"<<endl;
	}
};
class B
{
	public:
	void display()
	{
		cout<<"Class B"<<endl;
	}
};
class C : public A, public B
{
		void view()
	{
		display();
	}
};
int main()
{
	C c;
	c.display();
	return 0;
```

Vector India

```
    }
```

Output:

```
error: reference to 'display' is ambiguous
        display();
```

The above issue can be resolved by using the class resolution operator with the function. In the above example, the derived class code can be rewritten as:

```
class C: public A, public B
{
        void view()
        {
                A::display();  //Calling the display() function of class A.
                B::display(); // Calling the display() function of class B.

        }
};
```

An ambiguity can also occur in single inheritance.

Consider the following situation:

```
class A
{
        public:
        void display()
        {
                cout<<"Class A";
        }
};
class B
{
public:
        void display()
        {
                cout<<"ClassB";
        }
};
```

In the above case, the function of the derived class overrides the method of the base class. Therefore, call to the display() function will simply call the function

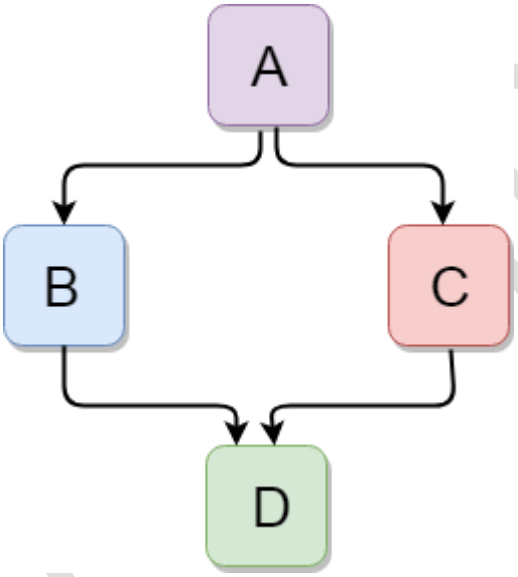
Vector India

defined in the derived class. If we want to invoke the base class function, we can use the class resolution operator.

```
int main()
{
        B b;
        b.display(); //Calling the display() function of B class.
        b.A:: display(); // Calling the display() function defined in A class.
}
```

C++ Hybrid Inheritance

Hybrid inheritance is a combination of more than one type of inheritance.



Let's see a simple example:

```
#include<iostream>
using namespace std;
class A
{
protected:
        int a;
public:
        void get_a()
        {
                cout<<"Enter the value of 'a' :"<<endl;
```

```
                cin>>a;
        }
};

class B : publicA
{
        protected:
                int b;
        public:
        void get_b()
        {
                cout<<"Enter the value of b:"<<endl;
                cin>>b;
        }
};
class C
{
protected:
int c;
public:
void get_c()
        {
        cout<<"Enter the value of c is:"<<endl;
        cin>>c;
        }
};

class D : public B, public C
{
protected:
int d;
public:
void mul()
{
        get_a();
        get_b();
        get_c();
        cout<<"Multiplication of a,b,c is:"<<a*b*c<<endl;
}
};
int main()
```

Vector India

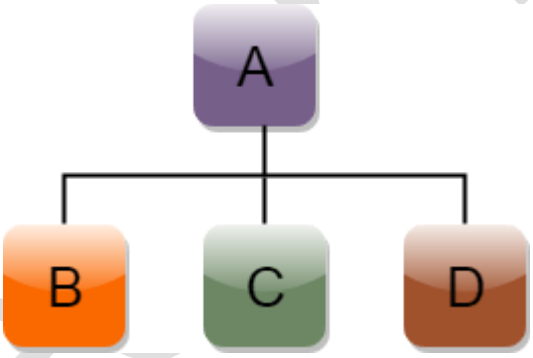
```
{
D d;
d.mul();
return 0;
}
```

Output:

```
Enter the value of 'a' :
10
Enter the value of 'b' :
20
Enter the value of c is :
30
Multiplication of a,b,c is : 6000
```

C++ Hierarchical Inheritance

Hierarchical inheritance is defined as the process of deriving more than one class from a base class.



Syntax of Hierarchical inheritance:

```
class A
{
        //body of the class A.
}
class B : public A
```

```
{
        //body of class B
}
class C : public A
{
        //body of class C.
}
class D : public A
{
        //body of class D.
}
```

Let's see a simple example:

```
#include<iostream>
using namespace std;
class Shape //Declaration of base class
{
public:
int a;
int b;
void get_data(int
1. public: n,int m)
{
a=n;
b=m;
}
};
class Rectangle : public Shape // inheriting Shape class
{
public:
int rect_area()
{
int result=a*b;
return result;
}
};
class Triangle : public Shape // inheriting Shape class
```

Vector India

```
{
public:
int triangle_area()
{
float result =0.5*a*b;
return result;
}
};
int main()
{
        Rectangle r;
Triangle t;
int length,breadth,base,height;
cout<<"Enter the length and breadth of a rectangle:"<<endl;
cin>>length>>breadth;
r.get_data(length,breadth);
int m=r.rect_area();
cout<<"Area of the rectangle is :"<<m<<endl;
cout<<"Enter the base and height of the triangle:"<<endl;
cin>>base>>height;
t.get_data(base,height);
float n=t.triangle_area();
cout<<"Area of the triangle is :"<<n<<endl;
return 0;
}
```

Output:

```
Enter the length and breadth of a rectangle:
23
20
Area of the rectangle is : 460
Enter the base and height of the triangle:
2
5
Area of the triangle is : 5
```

Vector India